SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

ONLINE QUIZ SYSTEM

Version 1.0 | 12 October 2026

Document information	Details
Project	Online Quiz System
Course / Department	Software Engineering
Prepared by	Avyukth KV (PES1UG24AM460)
Project team	Batchu Hari Nainika (PES1UG24AM461) — SRS | Avyukth KV (PES1UG24AM460) — SAD | Samvith Vaddiparthi (PES1UG24AM468) — STP
Faculty guide	Arpitha K
Baseline documents	Online Quiz System SRS v1.0; Software Test Plan v1.0
Status	Submission draft

Purpose: Describe the architecture, major components, interfaces, data model, security controls and design decisions for the Online Quiz System.

Document Control

Revision History

Version	Date	Author	Change summary
1.0	12 Oct 2026	Avyukth	Initial architecture and design specification

Approvals

Role	Name	Signature / Date
Team representative	Batchu Hari Nainika	Pending
Faculty guide	Arpitha K	Pending

Contents

1. Introduction....3
2. Document Overview....3
3. Architecture....4
4. Design....6
5. Appendices....11

1. Introduction

1.1 Purpose

This Software Architecture and Design (SAD) document describes the proposed architecture and implementation design of the Online Quiz System, translating SRS requirements into components, interfaces, data structures, security controls and runtime flows.

1.2 Scope

The system supports student registration/login, quiz discovery, timed quiz attempts, single-attempt enforcement, automatic scoring, score history and quiz-specific leaderboards. Faculty and administrators manage quizzes; administrators manage roles and review feedback. The design also includes CI quality checks.

1.3 Audience

Student developers, QA/test engineers, faculty evaluators, maintainers and administrators.

1.4 Definitions and Acronyms

Term	Meaning
SAD	Software Architecture and Design
SRS	Software Requirements Specification
STP	Software Test Plan
REST API	HTTP interface exchanging structured data
JWT	JSON Web Token for authenticated requests
RBAC	Role-Based Access Control
CI/CD	Continuous Integration / Continuous Delivery
RTM	Requirements Traceability Matrix

2. Document Overview

2.1 How to Use This Document

Section 3 describes architecture goals, stakeholders, pattern, components, technology, risks, traceability and security. Section 4 describes runtime flows, API contracts, data model, error handling and UI design. Appendices list open decisions and references.

2.2 Related Documents

Document	Use
Online Quiz System SRS v1.0	Requirements baseline
Online Quiz System STP v1.0	Testing strategy, test cases and acceptance evidence
Repository README.md	Setup, configuration and developer workflow

3. Architecture

3.1 Goals and Constraints

Goal / constraint	Design response
Separation of concerns	Separate React UI, Express API, business logic and persistence.
Quiz fairness	Enforce attempt limits, deadlines and scoring on the server.
Confidentiality	Never return answer keys or explanations in student responses.
Maintainability	Use modular routes/controllers/services/models and automated tests.
CI/CD	Run lint, tests, coverage, build and dependency audit in GitHub Actions.
Constraints	Browser application; JavaScript ecosystem; no specialist hardware required.

3.2 Stakeholders and Concerns

Stakeholder	Primary concerns
Students	Clear quiz flow, accurate timer and score, privacy
Faculty	Quiz authoring, publication and reliable scoring
Administrators	Role management, feedback review and access control
Developers	Modular code, testability and maintainability
Faculty evaluator	SRS traceability, working CI and demonstrable acceptance tests

3.3 High-Level Component View

Component	Responsibility
React + Vite frontend	Pages, forms, routing, timer display and API calls
Express REST API	Routes, request validation, authentication and response formatting
Application services	Quiz lifecycle, attempt rules, scoring, ranking and role administration
Mongoose models	Schema validation and database access
MongoDB	Users, quizzes, questions, attempts and feedback
GitHub Actions	Install, lint, test, coverage, build, audit and report artifacts

Logical request path: Browser UI → REST API → authentication/validation middleware → application service → Mongoose model → MongoDB. The API returns a response to the UI.

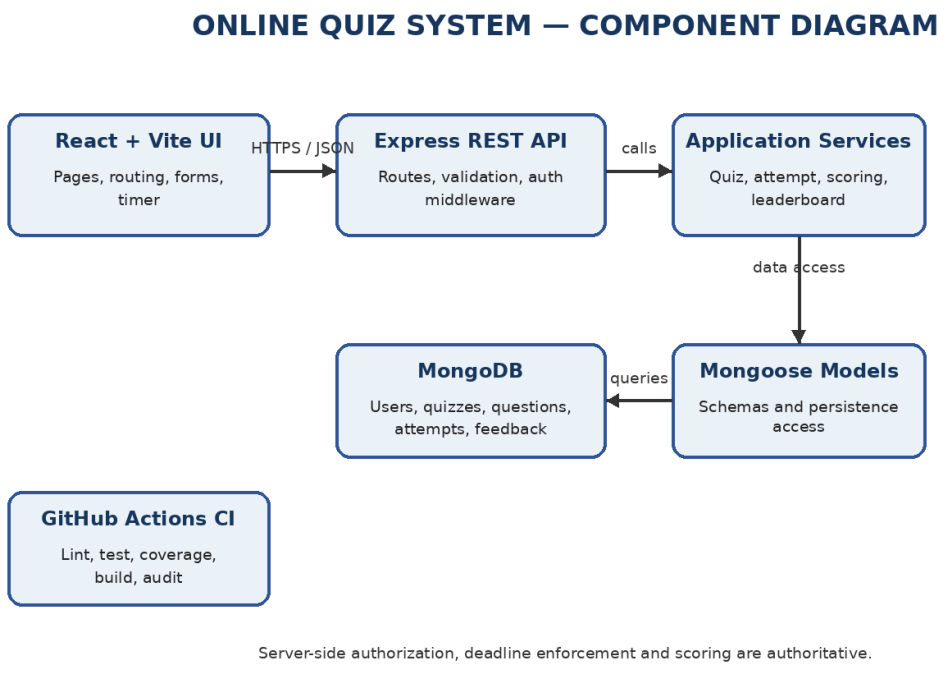


Figure 1. UML component view of the Online Quiz System.

3.4 Component Descriptions

Component	Responsibilities	SRS mapping
Authentication	Registration, login, password hashing, token validation and role checks	FR-001–004; SEC-001–002
Quiz module	Create/edit/delete/publish/unpublish and validate quizzes	FR-008–012
Discovery	List published quizzes and metadata	FR-005–007
Attempt	Create attempt, save answers, enforce deadline, prevent repeat attempts	FR-013–018; NFR-002–003
Scoring	Calculate score server-side; hide answer key	FR-019–020; SEC-003
History/leaderboard	Return own history and per-quiz ranking	FR-021–023
Feedback/admin	Store feedback, admin review and role changes	FR-024–026
CI workflow	Validate quality and package test evidence	FR-027–031; NFR-007; SEC-006

3.5 Architecture Pattern and Rationale

A layered modular monolith is proposed. React is a separate client; the backend exposes a REST API organized into routes/controllers, services, models and middleware. This is simpler for a student project than distributed microservices while preserving separation of concerns.

3.6 Technology Stack and Data Stores

Area	Proposed technology	Purpose
Frontend	React + Vite, React Router, Axios, plain CSS	Responsive UI and API calls
Backend	Node.js + Express	REST endpoints and business logic
Database	MongoDB + Mongoose	Persistence and schema validation
Authentication	bcryptjs + JWT	Password hashing and authenticated requests
Testing	Vitest, React Testing Library, Supertest	Unit, UI and API integration tests
Code quality	ESLint + Prettier	Lint and format checks
CI	GitHub Actions	Automated validation on push and pull request
Security	npm audit; optional CodeQL/gitleaks	Dependency and source/secret checks

3.7 Risks and Mitigations

Risk	Mitigation
Client manipulates timer	Use server deadline and validate every answer/submit request.
Duplicate attempts in concurrent requests	Use unique user/quiz constraint or atomic server-side check.
Answer key leaks	Use student-safe response objects that omit correct options and explanations.
Unauthorized changes	Apply role checks and resource ownership validation on API.
Test database unavailable	Use dedicated test DB or mongodb-memory-server.
Broken main branch	Require reviewed PRs and passing CI checks before merge.

3.8 Traceability to Requirements

SRS requirements	Component	Design evidence
FR-001–004; SEC-001–002	Authentication	Auth routes, password hash utility, JWT/RBAC middleware
FR-005–007	Discovery	Published quiz listing and metadata endpoint
FR-008–012	Quiz module	Quiz/question models and protected authoring endpoints
FR-013–018; NFR-002–003	Attempt module	Attempt model, server deadline, idempotent finalization
FR-019–023; SEC-003	Scoring/history/leaderboard	Scoring service and safe response schemas
FR-024–026	Feedback/admin	Feedback storage and admin-only routes
NFR-001	Express API / MongoDB	Indexes on quizId and (userId, quizId); compact student DTOs; load test TC-PERF-01 in STP
NFR-004–005	React frontend	Responsive layouts from 375 px; labelled forms with inline validation messages (Section 4.6)
SEC-004–005	API middleware / configuration	Request validation middleware; secrets from environment variables, .env not committed (Section 3.9)
FR-027–031; NFR-006–007; SEC-006	CI workflow	GitHub Actions, README and test artifacts

3.9 Security Architecture

The browser is not trusted to enforce roles, timers, scores or attempt limits. The API applies authorization and business rules on the server.

Threat / concern	Control
Spoofing	Password hashes, JWT validation and HTTPS if deployed
Broken access control	Role middleware and resource ownership checks
Answer-key disclosure	Keep answer keys server-side and omit from student API responses
Score/timer tampering	Calculate score server-side; validate deadline; reject post-finalization changes
Injection/malformed input	Validate IDs, types, ranges and request bodies
Secret exposure	Environment variables; do not commit .env or credentials
Dependency vulnerabilities	npm audit in CI with documented severity threshold

Use STRIDE as a review checklist: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege.

4. Design

4.1 Design Overview

React calls the Express REST API. The API validates requests, authenticates the caller, checks role and ownership, invokes a service and accesses MongoDB through Mongoose. The server is the source of truth for permissions, scoring, attempt status and deadlines.

4.2 Sequence Diagram Flows

4.2.1 Student Takes and Submits a Quiz

Step	Interaction
1	Student opens a published quiz in React.
2	Frontend requests quiz metadata; API returns details without answer keys.
3	Student starts; API returns the student's active attempt or creates one with start time/deadline; a finalized attempt is rejected (409).
4	Answer changes are sent; API checks attempt ownership and deadline before saving.
5	Student submits or time expires; API finalizes the attempt once.
6	Scoring service calculates score from the server-side answer key.
7	API returns score/status only; UI hides correct answers and explanations.

4.2.2 Faculty Creates and Publishes a Quiz

Step	Interaction

1	Faculty submits quiz metadata and questions.
2	API validates JWT, role, fields and ownership.
3	Quiz service validates options and correct-answer configuration.
4	Quiz is saved as a draft in MongoDB.
5	Faculty requests publication; required fields and questions are revalidated.
6	Quiz is published and becomes visible in student discovery.

The formal UML sequence diagrams below show lifelines and message flows for both scenarios.

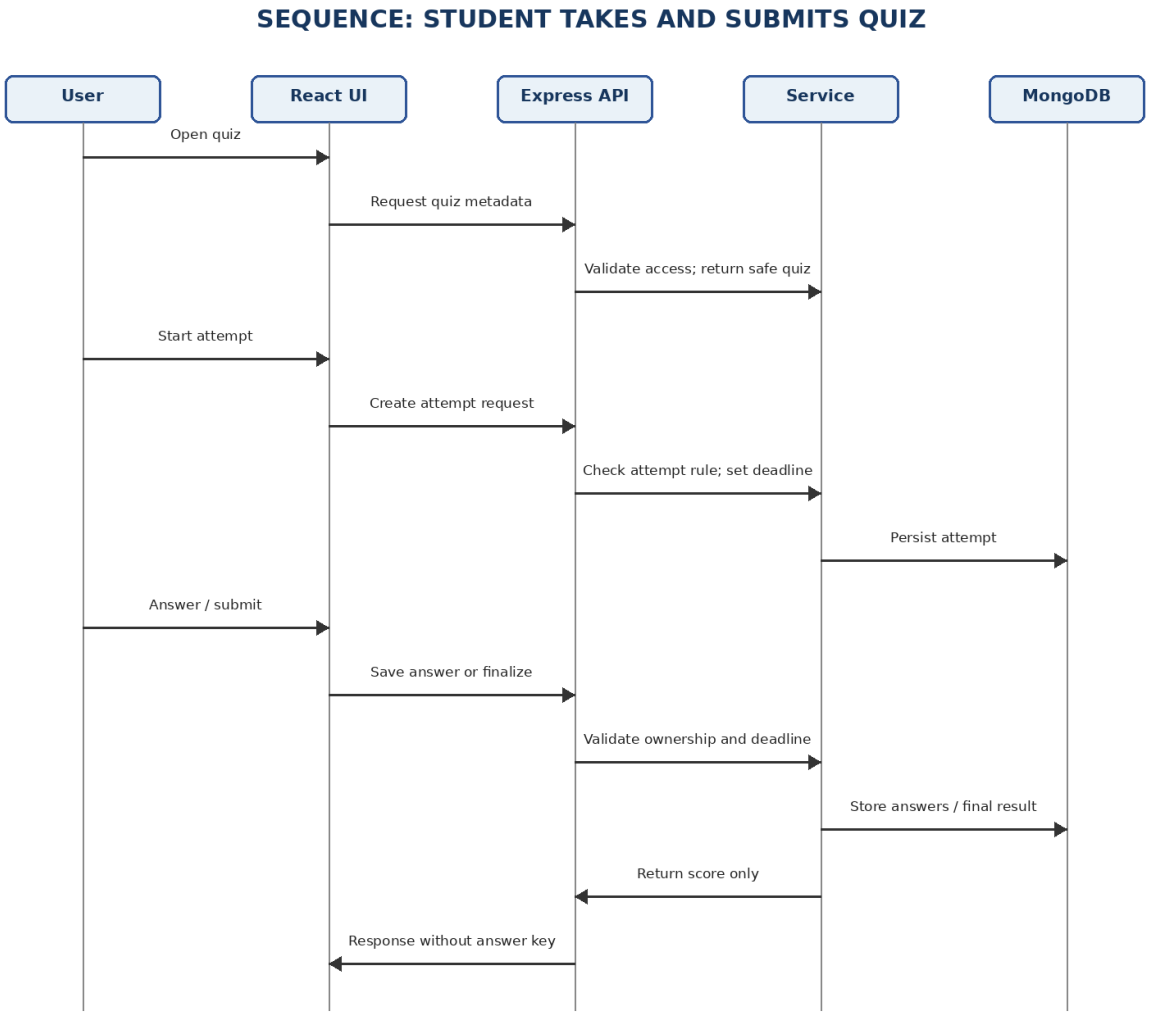


Figure 2. Student quiz-attempt sequence.

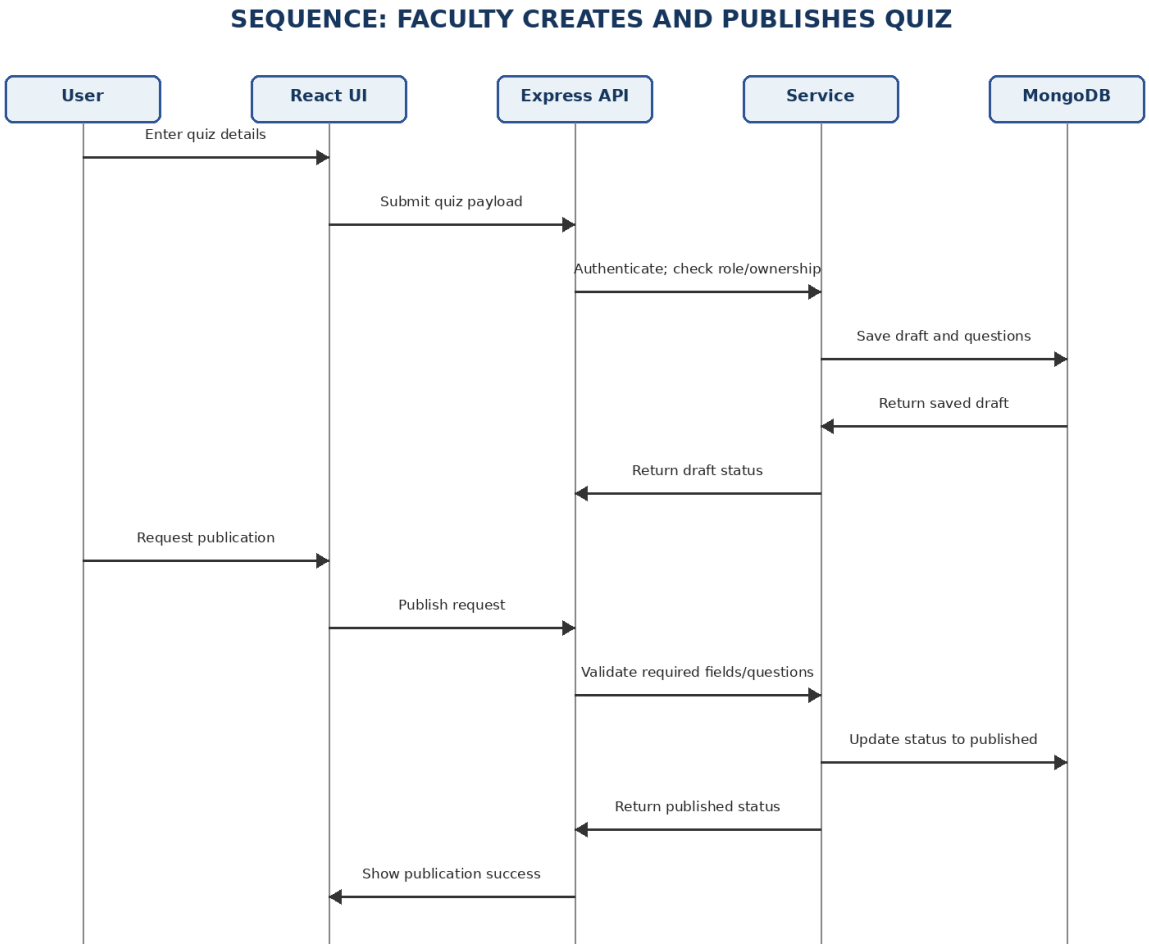


Figure 3. Faculty quiz-publication sequence.

4.3 API Design

The following REST contracts define the proposed API baseline. Any route or schema change made during implementation will be reflected in this table and in the README API documentation.

Method / endpoint	Access	Purpose
POST /api/auth/register	Public	Create student account; return safe profile or validation error
POST /api/auth/login	Public	Validate credentials and return token/session plus safe profile
POST /api/auth/logout	Authenticated user	Invalidate the caller's session so the logged-out token is rejected afterwards (FR-003)
GET /api/quizzes	Student	List published quizzes and metadata only
GET /api/quizzes/:quizId	Student	Quiz detail without answer keys/explanations
GET /api/manage/quizzes	Faculty/admin	List quizzes the caller may manage, including drafts
POST /api/quizzes	Faculty/admin	Create draft quiz
PATCH /api/quizzes/:quizId	Authorized faculty/admin	Edit quiz subject to ownership policy
DELETE /api/quizzes/:quizId	Authorized faculty/admin	Delete a draft quiz subject to ownership policy (FR-010)
POST /api/quizzes/:quizId/publish	Faculty/admin	Validate and publish complete quiz
POST /api/quizzes/:quizId/unpublish	Authorized faculty/admin	Remove a quiz from student discovery (FR-011)
POST /api/quizzes/:quizId/attempts	Student	Create an attempt, or return the student's active attempt; 409 if the attempt is already finalized
PATCH /api/attempts/:attemptId/answers	Attempt owner	Save answers while active and before deadline
POST /api/attempts/:attemptId/submit	Attempt owner	Finalize once and return score only
GET /api/me/scores	Student	Return current student's attempt history

GET /api/quizzes/:quizId/leaderboard	Authenticated user	Return ranking for this quiz
POST /api/feedback	Authenticated user	Store feedback
GET /api/admin/feedback	Administrator	Review submitted feedback
GET /api/admin/users	Administrator	List users and their roles for role management
PATCH /api/admin/users/:userId/role	Administrator	Change a user's role (FR-026)

4.3.1 Example Request and Response

POST /api/quizzes/:quizId/attempts

Request body: {} (quiz ID is in the path; identity comes from the authenticated session).

Example response: { "attemptId": "...", "status": "in_progress", "startedAt": "...", "deadlineAt": "..." }

Typical errors: 401 unauthenticated; 403 insufficient permission; 404 resource not found; 409 conflicting attempt/state; 422 invalid input.

4.3.2 Standard Error Handling

HTTP status	Meaning
400	Malformed request
401	Missing, invalid or expired authentication
403	Insufficient role or ownership
404	Resource not found
409	Conflicting state, such as an existing attempt
422	Invalid fields or business-rule failure
500	Unexpected error; do not expose stack trace

4.4 Data Model / Data Stores

Collection	Key fields	Notes
User	_id, name, email, passwordHash, role, tokenVersion, createdAt	Unique email; roles student/faculty/administrator; never return hash; tokenVersion invalidates tokens on logout
Quiz	_id, title, topic, durationSeconds, status, createdBy	createdBy references User; status draft/published
Question	_id, quizId, prompt, options, correctOption	Correct option is server-only and omitted from student DTOs
Attempt	_id, userId, quizId, answers, score, status, startedAt, deadlineAt, submittedAt	References User/Quiz; unique index on (userId, quizId) enforces one attempt record per user/quiz
Feedback	_id, userId, rating, message, createdAt	Admin-only review

Design decision: Questions are stored in a separate Question collection linked to Quiz by quizId. Index quizId for question lookup. Attempt creation must atomically enforce one attempt record per student/quiz; an active attempt can be resumed until deadline, while a finalized attempt cannot be restarted.

Session design: login issues a signed JWT containing the user ID, role and tokenVersion. Logout increments User.tokenVersion, so the authentication middleware rejects every token issued before logout (FR-003); tokens also expire after the configured lifetime.

4.5 Error Handling, Logging and Monitoring

- Return consistent JSON errors with safe messages; do not expose stack traces.
- Log request ID, timestamp, route, status and duration; never log passwords, tokens, secrets or answer keys.
- Track API errors, failed logins, submission failures, deadline rejections and CI status.
- Handle database errors and invalid IDs without crashing the server.

4.6 UX Design

Pages: Home, Register, Login, Quiz Topics, Take Quiz, My Scores, Quiz Leaderboard, About, Contact and Feedback. Faculty/admin users also access Quiz Management; administrators can review feedback and manage roles.

- Use consistent navigation, readable labels, validation messages and responsive layouts from 375 px upward.
- Show duration and countdown during an attempt; server deadline remains authoritative.
- After submission, display score/status only; never display correct answers or explanations.
- Provide loading, empty, success and error states for API-backed pages.

4.7 Open Issues and Next Steps

Open issue	Action required
Final API paths	Confirm routes against actual Express implementation.
Token lifetime and client storage	Agree the JWT expiry period and where the client keeps the token; document both in README.
Leaderboard privacy	Decide whether names or anonymized identifiers appear.
Editing published quizzes	Define behavior when attempts already exist.
Deployment	Optional unless course requires it; document HTTPS if deployed.

5. Appendices

5.1 Glossary

Term	Definition
Attempt	One student's attempt at a particular quiz
Answer key	Server-side correct option data used for scoring
DTO	Data Transfer Object: explicitly shaped API data
Idempotent finalization	Repeated submits do not create or alter multiple finalized attempts
Middleware	Express function validating requests before route logic
Published quiz	Quiz available to students

5.2 References

- Online Quiz System SRS v1.0, 12 October 2026.
- Online Quiz System STP v1.0, 12 October 2026.
- OWASP Top 10 and OWASP Web Security Testing Guide.
- Official Node.js, Express, React, Mongoose and GitHub Actions documentation.

5.3 Tools

Tool	Use
draw.io / PlantUML	UML component and sequence diagrams
Swagger / OpenAPI	REST API documentation if adopted
GitHub	Version control, PRs, issues and CI evidence
Vitest / React Testing Library / Supertest	Automated unit, UI and API integration tests